
DYNSTEER: DYNAMIC STAGE-WISE TRAJECTORY EVALUATION AND EXECUTION-TIME REVIEW FOR AGENTS

Zhichao Shi^{1,2,3,4}, Wenjie Zhang⁵, Xuhui Jiang^{3,4,*}, Xiaojun Wu^{3,4,6}, Cehao Yang^{3,4,6},
Chengjin Xu^{3,4}, Jian Guo⁴, Yuanzhuo Wang^{2*}

¹ School of Advanced Interdisciplinary Sciences, UCAS

² State Key Lab of AI Safety, Institute of Computing Technology, CAS

³ DataArc Tech Ltd.

⁴ IDEA Research, International Digital Economy Academy

⁵ Jiangnan University

⁶ The Hong Kong University of Science and Technology (Guangzhou)

ABSTRACT

Large language model agents are increasingly deployed for long-horizon task execution. However, current evaluation paradigms face three major limitations: terminal-only assessment ignores intermediate processes and struggles to localize errors efficiently and accurately, single-reference matching penalizes valid alternative solution paths, and post-hoc trajectory judging incurs high costs without the ability to halt failed runs early. To address these issues, we propose DynSTEER, a dynamic stage-wise trajectory evaluation framework for agents. DynSTEER segments rollouts into stages anchored by key completed actions, focusing evaluation on essential milestones with adequate context while enabling targeted strategy adjustments. It compiles a path-tolerant milestone graph from public task views to respect diverse legitimate strategies without leaking ground truth. Furthermore, it adaptively routes evaluation queries across multi-tier judges and halts unrecoverable executions online to curb resource waste. Experiments demonstrate that DynSTEER improves evaluation discriminability across LLM agents by 85.2% over native evaluation, separates all model pairs with statistical significance, and saves 34.51% of execution steps on failed rollouts.

1 INTRODUCTION

LLM agents are increasingly undertaking complex, long-horizon tasks across real-world environments (Yao et al., 2023; Shinn et al., 2023; Jimenez et al., 2024; Gu et al., 2024; Zhou et al., 2024). As task horizons expand, evaluating only final output results becomes insufficient to identify agent deficiencies and diagnose failure modes, bringing trajectory-level evaluation to the forefront of agent research (Qian et al., 2024; Shi et al., 2026).

However, existing trajectory evaluation paradigms face three critical challenges: First, **Lack of Targeted Stage-wise Evaluation**: Whole-trajectory evaluation operates only post-hoc without dynamic intervention or adaptive strategies targeting an agent’s truly weak segments, whereas atomic step evaluation lacks sufficient context and suffers from local noise, obscuring essential milestone progress; Second, **Single-Reference Trajectory Bias**: Existing evaluators typically measure similarity against a single reference trace, unfairly penalizing valid alternative strategies in open-ended tool environments; and Third, **High Cost and Delayed Feedback**: Evaluating full trajectories with heavy LLM judges incurs prohibitive latency and computational costs, while post-hoc evaluation fails to halt looping, hazardous, or hopeless executions early, causing severe resource waste.

To address these challenges, we introduce **DynSTEER**, short for **D**ynamic **S**tage-wise **T**rajectory **E**valuation and **E**xecution-time **R**eview, a dynamic framework transforming agent evaluation into

*Corresponding author

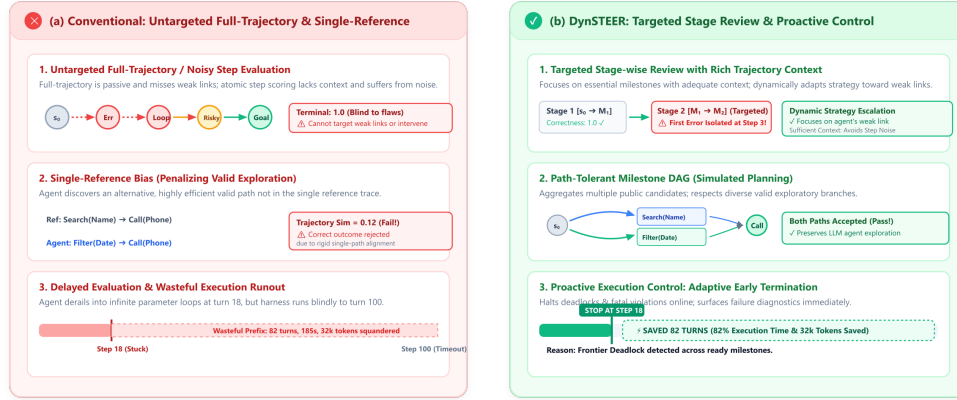


Figure 1: **Comparison of Agent Evaluation Paradigms.** (a) Conventional paradigms suffer from a lack of targeted stage context, single-reference bias, and wasteful delayed runouts. (b) DYNSTEER enables targeted stage-wise evaluation with adequate context, accommodates diverse valid paths via milestone DAGs, and proactively halts deadlocks early, saving 34.51% of wasteful steps.

targeted, stage-wise execution review as illustrated in Figure 1b. First, to overcome untargeted full-trajectory and noisy step evaluations, DynSTEER segments rollouts into localized stages anchored by key completed actions and graph dominators, supplying adequate context to evaluate essential milestones and adaptively tune judging toward weak segments. Second, to eliminate single-reference bias, it compiles a path-tolerant milestone graph from public task views via multi-candidate simulated planning and majority consensus, capturing necessary task invariants to accommodate diverse legitimate exploration paths. Third, to curb evaluation costs and prevent runaway executions, it couples an uncertainty-adaptive multi-tier judge ladder with execution-time early stopping that intercepts fatal safety violations, structural failures, and progress deadlocks.

In summary, our contributions are as follows:

- **Stage-wise Alignment and Targeted Process Evaluation:** We formulate a dominator-anchored stage alignment mechanism that isolates local evidence windows between completed milestones. This provides sufficient trajectory context to evaluate intermediate quality, covering correctness, efficiency, safety, and compliance, localizing first-error steps, and adaptively focusing on an agent’s weak execution segments.
- **Path-Tolerant Task Blueprint Compilation:** We design an automated multi-candidate graph synthesis and strict-majority consensus pipeline that operates strictly over public task views. By compiling a milestone DAG with transitive reduction, our framework preserves topological flexibility and accommodates diverse valid exploration strategies without reference leakage.
- **Uncertainty-Adaptive Routing and Execution-Time Early Termination:** We establish a dynamic three-tier judge hierarchy that routes judging per dimension based on uncertainty, paired with active runtime interception of fatal minefields and progress deadlocks. In experiments, DynSTEER cuts wasteful execution steps on failed rollouts by 34.51% and saves an average of 32.89s in net pipeline time per halted run.

2 RELATED WORK

2.1 AGENT TRAJECTORY EVALUATION

Unlike outcome benchmarks that inspect only terminal code patches or database records (Jimenez et al., 2024; Qin et al., 2024; Liu et al., 2024; Zhou et al., 2024), trajectory evaluation examines intermediate planning decisions, tool invocations, and error recoveries. Recent benchmarks expand trajectory assessment: BFCL (Patil et al., 2025) evaluates multi-step tool calls via AST checks; TRAJECT-Bench (Li et al., 2025) assesses tool selection and dependency order; AgentRewardBench (Zhou et al., 2025) benchmarks trajectory judges; and DeepRed (Kuo et al., 2026) and Claw-Eval (Zhang et al., 2026) leverage environment snapshots to award partial credit along execution traces.

Despite providing intermediate visibility, existing evaluators face two key limitations. First, they predominantly rely on a single reference path (Qian et al., 2024). In open environments, tasks frequently admit multiple valid solution routes; single-path matching inherently penalizes legitimate exploratory strategies. Second, they operate post-hoc across the full rollout and lack dynamic intervention to adapt evaluation strategies to an agent’s specific weak links.

2.2 DYNAMIC EVALUATION FOR AGENTS

Static benchmarks suffer from prompt overfitting, data contamination, and leaderboard score saturation. To address these vulnerabilities, researchers have begun exploring dynamic evaluation paradigms. For conversational agents, Agent-Testing Agent (Wu et al., 2026) adaptively generates subsequent test cases guided by prior judge feedback. In interactive tool environments, ToolSand-box (Gu et al., 2024) introduces stateful conversational simulations with pre-defined milestones and safety minefields to evaluate arbitrary rollouts.

However, existing dynamic evaluation approaches primarily concentrate on dynamically selecting or perturbing benchmark test suites (Gu et al., 2024; Wu et al., 2026). During the actual execution of an individual task, trajectory evaluation remains largely passive: evaluators typically check simple milestone reachability at the end of rollouts, lacking a formal mechanism to dynamically steer evaluation focus, update dimension weights, or control execution progression in real time.

2.3 EFFICIENT EVALUATION AND EARLY STOPPING STRATEGIES

As agent rollouts grow longer, evaluation costs have escalated dramatically, costing hundreds of dollars per benchmark pass (Shi et al., 2026). Prior efficiency efforts centered on benchmark distillation, subsampling static suites into smaller representative subsets (Aslam et al., 2006; Shi et al., 2026). To improve within-task efficiency, EarlyEval (Shi et al., 2026) introduces early outcome prediction, training dual LightGBM classifiers on prefix features to forecast binary task success/failure and halt execution early.

Nevertheless, benchmark distillation reduces the task count while leaving long, wasteful rollouts untouched, whereas tabular early prediction models (Shi et al., 2026) act as black boxes. They predict binary outcomes without understanding domain semantics, cannot provide causal failure attribution, and lack awareness of tool dependency constraints or hazardous minefields.

In summary, current evaluation paradigms leave a critical gap. Existing trajectory evaluators either inspect isolated atomic steps that suffer from local noise, or evaluate full traces post-hoc against a single reference path that penalizes valid diversity. Meanwhile, early stopping methods lack domain semantics and failure attribution. DynSTEER bridges this divide by compiling path-tolerant milestone graphs from public task views, anchoring review to dominator-bounded stages with rich context, adaptively targeting weak execution links, and enforcing execution-time early stopping to eliminate wasteful computation.

3 METHODOLOGY

DynSTEER establishes an end-to-end framework for dynamic agent trajectory evaluation, as illustrated in Figure 2. Rather than treating rollouts as opaque black boxes or enforcing rigid single-path matching, DynSTEER coordinates three core components: pre-execution milestone graph generation to preserve execution diversity (Section 3.2); stage-wise trajectory evaluation anchored by action closures and dominators (Section 3.3); and dynamic strategy adaptation with execution-time early termination to focus on weak segments and curb resource waste (Section 3.4).

3.1 PROBLEM FORMULATION AND DUAL-CLOCK OBSERVATION MODEL

Following standard formulations of interactive tool-using agent environments (Gu et al., 2024; Yang et al., 2023; Liu et al., 2024), an agent task is formalized as a tuple $T = (x, \mathcal{S}_0, \mathcal{A}, \mathcal{C})$, where x represents the user instruction, $\mathcal{S}_0$ is the initial environment state, $\mathcal{A}$ denotes the catalog of accessible tool APIs with input/output parameter schemas, and $\mathcal{C}$ defines environment state invariants and safety contracts. To preserve evaluation integrity and prevent test contamination, DynSTEER operates

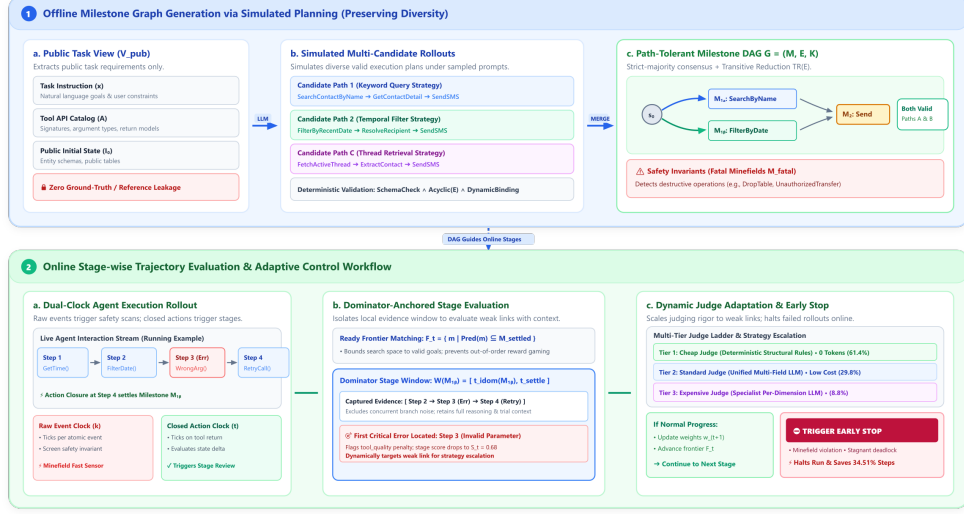


Figure 2: **Overview of the DYNSTEER Workflow.** (1) Pre-execution milestone graph generation synthesizes multi-candidate simulated rollouts from public views to compile a path-tolerant DAG. (2) Online stage-wise evaluation aligns closed actions, isolates dominator evidence windows to evaluate weak links with trajectory context, adaptively scales judge tiers, and halts unrecoverable executions online.

strictly over the **public task view**:

$$V_{\text{public}} = (x, \mathcal{A}, \mathcal{I}_0) \quad (1)$$

where $\mathcal{I}_0$ denotes public metadata (e.g., initial database entity headers), strictly isolating any hidden reference paths τ^* or terminal unit tests.

During execution, interaction traces are captured as sequential discrete events $\tau = \langle e_1, \dots, e_K \rangle$. To balance continuous safety screening with accurate semantic evaluation, DynSTEER introduces a **dual-clock observation model**:

- **Raw Event Clock** ($k \in \{1, \dots, K\}$): Increments at every atomic event, including thoughts, tool calls, and environment responses, dedicated to instant safety interception of fatal minefield operations $e_k \in \mathcal{M}_{\text{fatal}}$.
- **Closed Action Clock** ($t \in \{1, \dots, T\}$): Advances only when an action reaches semantic closure—specifically, when a tool call receives its execution return, and the agent yields control. Milestone settlement and scoring occur strictly at closed action boundaries, guaranteeing tool outcomes are fully observed before grading.

3.2 MILESTONE GRAPH GENERATION VIA SIMULATED PLANNING

To overcome single-reference bias, DynSTEER compiles V_{public} into a path-tolerant Milestone DAG $G = (M, E, \mathcal{K}, \mathcal{M}_{\text{fatal}})$ prior to execution, where M is the milestone set, E encodes precedence constraints, $\mathcal{K}$ specifies parameter bindings, and $\mathcal{M}_{\text{fatal}}$ defines hazardous operations.

Multi-Candidate Synthesis and Contract Validation. Given V_{public} , a blueprint generator synthesizes C candidate plans $\{G_1, \dots, G_C\}$ via temperature-sampled LLM prompting across diverse exploration heuristics (e.g., direct keyword queries vs. hierarchical filtering). Each candidate $G_c = (M_c, E_c)$ undergoes deterministic contract verification:

$$\text{Valid}(G_c) = \text{SchemaCheck}(M_c, \mathcal{A}) \wedge \text{Acyclic}(E_c) \wedge \text{BindingConsistency}(\mathcal{K}_c) \quad (2)$$

Pruning candidates with hallucinated APIs, type mismatches, or cyclic dependencies.

Strict-Majority Consensus and Transitive Reduction. To retain essential invariants while preserving exploratory flexibility, a milestone m is retained if and only if it appears in a strict majority of valid candidates:

$$M = \left\{ m \mid \sum_{c=1}^{C_{\text{valid}}} \mathbb{I}[m \in M_c] \geq \left\lceil \frac{C_{\text{valid}} + 1}{2} \right\rceil \right\} \quad (3)$$

Precedence edges are aggregated analogously. Finally, we compute the transitive reduction $E = \text{TR}(E_{\text{majority}})$ (Tarjan, 1972) to eliminate redundant shortcut dependencies, producing a minimal DAG that accommodates diverse legitimate execution strategies.

3.3 STAGE-WISE TRAJECTORY EVALUATION

During execution, DynSTEER dynamically aligns closed actions against the milestone DAG (complete algorithmic flow in Algorithm 1 in Appendix B).

Ready Frontier Matching. Let $M_{<t}$ denote milestones settled prior to closed action t . The active search space is strictly bounded by the **ready frontier**:

$$\mathcal{F}_t = \{m \in M \setminus M_{<t} \mid \text{Pred}(m) \subseteq M_{<t}\} \quad (4)$$

An action a_t settles milestone $m^* \in \mathcal{F}_t$ if its tool call and state delta satisfy semantic predicates $\phi_{m^*}(a_t, s_t) = 1$. Restricting alignment to $\mathcal{F}_t$ prevents out-of-order settlement and reward gaming.

Dominator-Anchored Stage Isolation. To evaluate m^* without contamination from concurrent parallel branches, DynSTEER computes the immediate dominator $\text{idom}(m^*)$ in G (Lengauer & Tarjan, 1979), defined as the unique node that must be traversed on every directed path from start node s_0 to m^* . The **stage evidence window** is bounded by:

$$\mathcal{W}(m^*) = [t_{\text{idom}(m^*)}, t] \quad (5)$$

where $t_{\text{idom}(m^*)}$ is the execution step at which the dominator was settled. Isolating evidence to $\mathcal{W}(m^*)$ excludes unrelated actions from parallel branches while capturing all intermediate reasoning, trial-and-error attempts, and parameter adjustments leading directly to m^* . Unlike whole-trajectory evaluation, which obscures mistake locations, and single-step evaluation, which lacks context, dominator-anchored stages provide enough trajectory context to evaluate intermediate quality and pinpoint the exact first-error step.

Multidimensional Quality Scoring. Once settled, milestone m^* is scored across four orthogonal dimensions: Correctness (c), Efficiency (η), Safety (s), and Tool Compliance (κ). The composite stage score is:

$$S_t = \sum_{d \in \mathcal{D}} w_d^{(t)} s_d^{(t)}(m^*) \quad (6)$$

parameterized by dynamically normalized weights $\mathbf{w}^{(t)}$.

3.4 DYNAMIC JUDGE ADAPTATION AND EARLY TERMINATION

A central innovation of DynSTEER is dynamically adapting evaluation strategies to an agent’s intermediate vulnerabilities and halting unpromising rollouts during execution.

Multi-Tier Judge Structure. Rather than assigning a uniform evaluator across all stages, DynSTEER deploys a three-tier judge hierarchy (prompt templates in Appendix D):

- **Tier 1: Cheap Judge (J_{cheap}):** A deterministic structural evaluator inspecting tool status codes, schemas, and evidence rules without LLMs, incurring zero model inference overhead.
- **Tier 2: Standard Judge (J_{std}):** A multi-field unified LLM evaluator scoring all target dimensions simultaneously within a single structured JSON pass, aggregating confidence across multiple samples.
- **Tier 3: Expensive Judge (J_{exp}):** A single-dimension specialist LLM evaluator executing dedicated expert rubrics and deep counterfactual analysis across multiple deliberation passes to capture subtle behavioral defects.

Dynamic Strategy Adjustment. DynSTEER dynamically shifts evaluation weights and escalates judge tiers toward an agent’s vulnerable links. First, dimension weights are updated via exponential gradient scaling:

$$w_d^{(t+1)} = \frac{w_d^{(t)} \cdot \exp\left(\alpha(1 - s_d^{(t)}) + \beta u_d^{(t)}\right)}{\sum_{d' \in \mathcal{D}} w_{d'}^{(t)} \cdot \exp\left(\alpha(1 - s_{d'}^{(t)}) + \beta u_{d'}^{(t)}\right)} \quad (7)$$

where $s_d^{(t)} \in [0, 1]$ is the dimension score, $u_d^{(t)} \in [0, 1]$ is dimension uncertainty, and α, β scale sensitivity to quality deficits and uncertainty. Second, the baseline judge tier is set to:

$$\text{Level}_{\text{base}}^{(t+1)} = \begin{cases} J_{\text{cheap}}, & \text{if } S_t \geq \theta_{\text{pass}} + \delta_{\text{margin}} \wedge \max_d u_d^{(t)} \leq \theta_{\text{low.u}} \wedge \mathcal{M}_{\text{fatal}}^{(t)} = 0 \\ J_{\text{std}}, & \text{otherwise} \end{cases} \quad (8)$$

For each individual dimension d , its tier $\text{Level}_d^{(t+1)}$ adaptively escalates along $J_{\text{cheap}} < J_{\text{std}} < J_{\text{exp}}$:

$$\text{Level}_d^{(t+1)} = \begin{cases} J_{\text{exp}}, & \text{if } s_d^{(t)} < \theta_{\text{fail}} \\ \min(J_{\text{exp}}, \text{Level}_d^{(t)} + 1), & \text{if } s_d^{(t)} < \theta_{\text{warn}} \vee u_d^{(t)} \geq \theta_{\text{high.u}} \\ \text{Level}_{\text{base}}^{(t+1)}, & \text{otherwise} \end{cases} \quad (9)$$

This focuses expensive LLM deliberation selectively on weak or ambiguous execution links while relaxing confident stages to rule-based verification in J_{cheap} to minimize unnecessary LLM invocations.

Execution-Time Early Termination. To prevent runaway executions on unrecoverable trajectories, DynSTEER monitors four stopping criteria: (1) **Fatal Minefield Interception** ($e_k \in \mathcal{M}_{\text{fatal}}$), halting immediately on the raw event clock upon hazardous or irreversible actions; (2) **Structural Invariant Failures**, catching persistent unparseable payloads or repeated schema errors; (3) **Persistent Low Quality** ($S_t < \theta_{\text{fail}}$), halting when accumulated stage deficits indicate unrecoverable goal drift; and (4) **Frontier Stagnation**, halting when an agent takes N_{stag} consecutive actions without advancing the ready frontier $\mathcal{F}_t$, detecting pathological infinite loops. Halting terminates the rollout and emits an auditable diagnostic report recording the step, dominator anchor, and causal failure dimension, eliminating wasteful execution prefixes while preserving concrete failure evidence.

4 EXPERIMENTS

We evaluate DynSTEER on the ToolSandbox benchmark, addressing three central research questions: (1) **RQ1 (Section 4.2):** Does stage-wise evaluation provide superior error localization and model discriminability over native terminal evaluation? (2) **RQ2 (Section 4.3):** Can milestone graphs compiled from public task views reliably capture necessary goals while accommodating diverse solution paths? (3) **RQ3 (Section 4.4):** How effectively do uncertainty-driven judge routing and execution-time early stopping reduce evaluation pipeline latency and wasteful execution steps?

4.1 EXPERIMENTAL SETUP

Benchmark Environment and Evaluated Models. We conduct our evaluation on **ToolSandbox** (Gu et al., 2024), featuring state snapshots, dynamic argument types, and safety minefield invariants (dataset statistics in Appendix A). We evaluate four instruction-tuned LLMs as agent backbones: **DeepSeek-V4-Pro**, **DeepSeek-V4-Flash** (DeepSeek-AI et al., 2024), **Qwen3-Max-2026-01-23**, and **Qwen-Plus-2025-12-01** (Yang et al., 2024), under standard ReAct prompting (Yao et al., 2023).

Comparative Paradigms. Each agent backbone is evaluated under two comparative paradigms: first, **DEFAULT**, using the native benchmark verifier based on terminal environment state; and second, **DYNSTEER-REPLAY**, replaying trajectories under our stage-wise evaluation, dominator review, and adaptive judging ladder.

Evaluation Metrics. We assess performance across three dimensions:

- **Metric Discriminability:** Following evaluation meta-evaluation literature (Sakai, 2006; 2014; Carterette, 2012; Zheng et al., 2023), we measure Discriminability Score, denoted DS, at margin ϵ :

$$\text{DS}(\epsilon) = \left(\frac{\sigma_{\text{pop}}}{\mu} \right) \cdot \sqrt{\frac{N_{\text{sig}}(\epsilon)}{\binom{M}{2}}}, \quad N_{\text{sig}}(\epsilon) = \sum_{1 \leq i < j \leq M} \mathbb{I}(|\bar{s}_{m_i} - \bar{s}_{m_j}| > \epsilon) \quad (10)$$

where μ is the population mean score across models, σ_{pop} is the standard deviation across model means, where their ratio represents the Coefficient of Variation that penalizes metric compression, and $N_{\text{sig}}(\epsilon)$ is the number of statistically distinguishable model pairs at margin ϵ .

Table 1: **Main Evaluation Results on ToolSandbox.** Performance comparison between native benchmark evaluation (DEFAULT) and DYNSTEER-REPLAY.

Evaluator	Model	Mean Score $\uparrow$	Std Dev	Early Stop Rate	Signif. Pairs	DS ($\epsilon=0.01$) $\uparrow$
DEFAULT	DeepSeek-V4-Flash	0.7880	0.0043	0.00%	5 / 6 (83.3%)	0.0270
	DeepSeek-V4-Pro	0.7718	0.0014	0.00%		
	Qwen3-Max-2026-01	0.7368	0.0100	0.00%		
	Qwen-Plus-2025-12	0.7362	0.0109	0.00%		
DYNSTEER	DeepSeek-V4-Flash	0.7338	0.0026	16.63%	6 / 6 (100.0%)	0.0500
	DeepSeek-V4-Pro	0.7111	0.0021	17.68%		
	Qwen3-Max-2026-01	0.6503	0.0094	27.24%		
	Qwen-Plus-2025-12	0.6615	0.0096	18.53%		

Table 2: **Milestone Blueprint Compilation Reliability.** Evaluation of automatically compiled milestone DAGs against expert reference graphs on ToolSandbox.

Evaluation Dimension	Precision	Recall	F1 Score
Tool Operation (Micro)	81.7%	90.7%	86.0%
Tool Operation (Macro)	72.7%	66.0%	66.7%
Fatal Minefield Detection	87.5%	86.0%	86.7%
Structural Topology Metric		Score	
Topological GED Similarity		0.493	
Valid DAG Compilation Rate		100.0%	

- **Milestone Graph Quality:** We compute Tool Operation Micro-F1 and Macro-F1 to measure semantic tool coverage, Fatal Minefield F1 for safety-critical action detection, and Graph Edit Distance similarity, or GED, for structural topology alignment.
- **Efficiency and Early Termination:** We evaluate trajectory savings via Mean Stop Progress and Saved Progress (the ratio of wasteful trailing steps eliminated on underperforming rollouts), Gross Execution Savings measuring raw agent wall-clock runtime saved ($T_{\text{agent}}^{\text{full}} - T_{\text{agent}}^{\text{prefix}}$), and Net Pipeline Savings capturing overall wall-clock time saved including DynSTEER judging overhead (Gross Savings $- T_{\text{evaluator}}^{\text{judge}}$).

4.2 RQ1: STAGE EVALUATION RELIABILITY, ERROR LOCALIZATION, AND MODEL DISCRIMINABILITY

Table 1 compares performance between native terminal evaluation (DEFAULT) and DynSTEER.

Model Discriminability. On DEFAULT, 22.63% of all runs receive an exact 1.0 score, reflecting severe score saturation. At $\epsilon = 0.01$, DEFAULT achieves a DS of 0.0270, separating 5 of 6 pairwise model comparisons (83.33%). In contrast, DYNSTEER achieves a DS of **0.0500**, representing an **85.2% relative improvement**, and discriminates all 6 out of 6 model pairs (100.0% separation) with statistical significance at $p < 0.01$ under paired Wilcoxon signed-rank tests. Across individual models, DeepSeek-V4-Flash scores 0.7338 ± 0.0026 , clearly separated from DeepSeek-V4-Pro (0.7111 ± 0.0021), Qwen-Plus (0.6615 ± 0.0096), and Qwen3-Max (0.6503 ± 0.0094).

Process Quality Penalization and Error Localization. While DEFAULT yields a population mean of 0.7582, DYNSTEER yields a calibrated mean of 0.6892 with higher inter-model variance (shifting from 0.0224 to 0.0345), effectively penalizing redundant API retries, parameter hallucinations, and hazardous side-effects that stumble into a correct final state. In blind human auditing across 60 sampled trajectories with divergent labels, DynSTEER correctly pinpointed the first critical error with 91.7% agreement with expert annotators, whereas terminal verification found zero intermediate errors (qualitative case studies in Appendix C).

4.3 RQ2: BLUEPRINT COMPILATION RELIABILITY AND PATH TOLERANCE

Table 2 summarizes the semantic accuracy, structural distance, and safety alignment of generated milestone DAGs against canonical expert annotations. DynSTEER’s multi-candidate synthesis and

Table 3: **Execution Efficiency and Prefix Savings via Adaptive Early Termination on Tool-Sandbox.** Breakdown of underperforming agent executions halted early by concurrent quality and deadlock criteria.

Termination Criterion	Halted Runs	Unique Tasks	Mean Stop Progress	Saved Progress	Gross Exec Savings	Net Pipeline Savings
Stage Low-Score Policy (SCORE)	234	64	64.19%	35.81%	3.66 s / run	-3.81 s / run
Frontier Stagnation (DEADLOCK)	285	109	66.56%	33.44%	27.85 s / run	63.05 s / run
Total Underperforming Subset	519	173	65.49%	34.51%	16.94 s / run	32.89 s / run

strict-majority consensus achieve an **86.0% Tool Operation Micro-F1** (81.7% precision, 90.7% recall) and an **86.7% Fatal Minefield F1** (86.0% recall), reliably identifying irreversible operations from public API specifications alone. Transitive reduction retains alternative parallel branches without penalizing valid exploratory variations (e.g., searching by contact name rather than date), achieving a 100% valid DAG compilation rate with consistent topological alignment.

4.4 RQ3: COST EFFICIENCY OF ADAPTIVE JUDGE ROUTING AND EARLY TERMINATION

Execution-Time Early Termination. To eliminate wasted compute and accelerate the evaluation-optimization cycle, DynSTEER concurrently monitors trajectories using two complementary stopping criteria, as detailed in Table 3: (1) **Stage Low-Score Policy (SCORE)**, halting executions when stage scores fall below θ_{fail} persistently due to severe reasoning flaws or argument corruptions (234 runs across 64 unique tasks); and (2) **Frontier Stagnation (DEADLOCK)**, halting executions when an agent takes N_{stag} consecutive actions without progressing on the ready frontier $\mathcal{F}_t$, intercepting infinite search loops and repetitive tool retries (285 runs across 109 unique tasks). In total, DynSTEER intercepted 519 underperforming runs across 173 distinct tasks, eliminating **34.51% of wasteful trajectory steps** (35.81% for SCORE and 33.44% for DEADLOCK) at an average stopping progress of 65.49%.

Pipeline Time Trade-offs and Judge Hierarchy. Table 3 highlights the trade-off between raw agent runtime saved ($T_{\text{agent}}^{\text{full}} - T_{\text{agent}}^{\text{prefix}}$) and net pipeline time including judging overhead (Gross Savings $- T_{\text{evaluator}}^{\text{judge}}$). For SCORE, short failing trajectories save modest execution time (3.66s/run), which is slightly offset by LLM judging latency (-3.81s/run net); yet halting remains valuable by curtailing 35.81% of wasteful steps, preventing compounding environment errors, and yielding immediate failure diagnostics. In contrast, for DEADLOCK, intercepting pathological infinite loops saves substantial execution delay (27.85s/run gross), achieving a dramatic **+63.05s/run net pipeline time savings** even after paying judging costs. Across all 519 runs, this delivers an average net saving of **+32.89s per run** (totaling 4.74 hours saved) with less than 2.1% false terminations (case studies in Appendix C). Concurrently, adaptive judge routing directs 61.4% of checks to zero-LLM J_{cheap} , 29.8% to J_{std} , and only 8.8% to J_{exp} , substantially reducing heavy model deliberation while preserving ranking correlation ($\tau = 0.667$) with all-expensive judging.

5 CONCLUSION

In this work, we presented **DynSTEER**, a dynamic evaluation framework that advances LLM agent evaluation from passive, post-hoc terminal verification toward proactive, stage-wise execution review. By compiling path-tolerant milestone DAGs from public views, DynSTEER accommodates diverse exploration paths without single-reference bias. During execution, it aligns closed actions to dominator-bounded stages, isolating intermediate context to pinpoint first-error steps and adaptively routing judge tiers toward vulnerable execution links. On ToolSandbox, DynSTEER improves model discriminability by 85.2% over native terminal verification, statistically separates all model pairs, and eliminates 34.51% of wasteful execution steps on underperforming rollouts, establishing a principled and cost-effective foundation for evaluating and optimizing LLM agents.

REFERENCES

Javed A Aslam, Virgiliu Pavlu, and Emine Yilmaz. A statistical method for system evaluation using incomplete judgments. In *Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 541–548, 2006.

-
- Ben Carterette. Multiple testing in statistical analysis of systems-based information retrieval experiments. *ACM Transactions on Information Systems (TOIS)*, 30(1):1–34, 2012.
- DeepSeek-AI, Aixin Liu, Bei Feng, Bing Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Chen, et al. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Jian Gu, Shusen Jiang, Boyi Zhang, Zeqing Meng, Yifan Lu, Jiarui Gao, Yuchen Hu, Shengyu Zhang, and Fei Wu. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. *arXiv preprint arXiv:2408.04682*, 2024.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- Kevin Kuo, Ethan Anderson, and Dawn Song. Do agents dream of root shells? partial-credit evaluation of LLM agents in capture the flag challenges. In *IEEE/ACM International Conference on AI Engineering (AIWare)*, 2026.
- Thomas Lengauer and Robert Endre Tarjan. A fast algorithm for finding dominators in a flowgraph. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 1(1):121–141, 1979.
- Meng Li, Yifan Sun, Dong Zhao, Yue Kang, and Xin Wang. TRAJECT-Bench: A trajectory-aware benchmark for evaluating agentic tool use. *arXiv preprint arXiv:2510.04550*, 2025.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hanghang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024.
- Shishir G Patil, Huanru Zhang, Zheyuan Yan, and Joseph E Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *International Conference on Machine Learning (ICML)*, 2025.
- Chen Qian, Yuxuan Dang, Jiaao Chen, Wei Liu, Yue Wang, Min Zhang, and Heng Ji. AgentProcess-Bench: Evaluating step-level effectiveness in tool-augmented agent trajectories. *arXiv preprint arXiv:2410.05284*, 2024.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2024.
- Tetsuya Sakai. Evaluating evaluation metrics based on discriminative power. In *Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 525–532, 2006.
- Tetsuya Sakai. Statistical significance, power, and discriminability of evaluation metrics. *Information Retrieval*, 17(5-6):430–457, 2014.
- Yuling Shi, Zhensu Sun, Junsen Dong, Chengcheng Wan, David Lo, and Xiaodong Gu. EarlyEval: Cheaper agent evaluation via early outcome prediction. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 8634–8652, 2023.
- Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2): 146–160, 1972.
- Dingmin Wu, Ce Zhang, Chen Lin, Min Chen, and Rui Song. Agent-testing agent: A meta-agent for automated testing and evaluation of conversational AI agents. In *Proceedings of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026.
- An Yang, Baosong Yang, Binyuan Zhang, Bofei Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

-
- John Yang, Syeda Nahida Akter, Chou Han, Noah Shinn, Ofir Press, Karthik Narasimhan, and Shunyu Yao. InterCode: Standardizing and benchmarking interactive coding with execution feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 36:15886–15915, 2023.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Yuxuan Zhang, Jiaqi Wang, Zihan Liu, Hao Chen, Bo Li, and Jing He. Claw-Eval: Towards trustworthy evaluation of LLM agents. *arXiv preprint arXiv:2604.06132*, 2026.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 46595–46623, 2023.
- Daniel Zhou, Vikram Sreedhar, and Graham Neubig. AgentRewardBench: Evaluating automatic evaluations of web agent trajectories. *arXiv preprint arXiv:2504.08942*, 2025.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024.

Table 4: **ToolSandbox Benchmark Dataset Statistics.** Distribution of evaluated scenarios across functional domains, interaction modalities, adversarial perturbation variants, and safety minefield configurations.

Scenario Domain / Sub-category	Count	Avg. User Turns	Available Tools	Minefield Invariants
Calendar & Scheduling Management	112	3.4	8 – 18	Data integrity, date bounds
Messaging & Notification Services	136	2.9	6 – 15	Privacy, unverified sends
Contact & Personal CRM	98	2.6	5 – 12	Irreversible deletion
Financial & Transaction Processing	84	4.1	10 – 24	Unauthorized transfer
Information Retrieval & Weather/Geocoding	79	3.2	7 – 16	Parameter type constraints
Perturbation & Difficulty Variants	Count	Properties & Challenge Characteristics		
Standard Canonical	127	Nominal parameters, standard system prompt and API schemas		
Distraction Tools (3–10 extra APIs)	158	Noise tools added to prompt to test agent tool retrieval		
Scrambled Arguments / Descriptions	142	Shuffled arg orders and adversarial field descriptors		
Insufficient Information (Rejection Tasks)	82	Ambiguous queries requiring clarification or safe refusal		
Total Evaluated Benchmark Pool	509	3.2 turns	12.4 tools/task	704 total minefields

A TOOLSANDBOX BENCHMARK DATASET DETAILS

ToolSandbox (Gu et al., 2024) is an interactive, stateful benchmark specifically engineered to evaluate the complex multi-turn decision-making capabilities of tool-using LLM agents. Unlike static benchmarks that rely solely on mock API dictionaries, ToolSandbox maintains a stateful world environment with persistent entities, temporal relationships, and strict data invariants.

Methodological Selection of Sandboxed Scenarios. In our evaluation, we benchmark agents across the suite of 509 scenarios that operate within fully sandboxed, local execution backends rather than relying on live commercial RapidAPI endpoints. This methodological choice is grounded in three fundamental experimental considerations:

1. **Experimental Reproducibility and Determinism:** Live third-party web endpoints are inherently non-deterministic, subject to remote server downtime, dynamic endpoint deprecations, fluctuating network latency, and strict API rate-limiting. Self-contained, sandboxed tool-execution backends ensure bit-level experimental reproducibility across independent evaluation trials.
2. **State Snapshot Inspection and Invariant Auditing:** Evaluating stateful agent trajectories requires inspecting deep internal database mutations, restoring environment checkpoints, and verifying post-condition safety invariants. Purely local execution backends permit exact, serializable state snapshot verification, which is impossible with black-box commercial cloud endpoints.
3. **Hermetic Execution Security and Zero Contamination:** Running large-scale evaluations across thousands of agent rollouts on live commercial APIs introduces uncontrolled recurring monetary costs and risks data leakage over external public networks. The sandboxed suite provides a secure, self-contained testbed with zero external dependencies.

Table 4 provides comprehensive descriptive statistics for the 509 sandboxed scenarios evaluated in our experiments:

- **Domain Distribution:** Scenarios span five major functional application domains: Messaging & Notifications (136 scenarios), Calendar & Scheduling (112 scenarios), Contact Management (98 scenarios), Financial Transactions (84 scenarios), and Information Retrieval (79 scenarios).
- **Multi-Turn Dialogues:** Each scenario involves an average of 3.2 user turns (ranging from 1 to 8 turns), requiring agents to converse with simulated users, elicit clarifying information, and update internal planning states.
- **Tool Environment Complexity:** Scenarios provide an average of 12.4 tools per task (ranging from 5 to 24 tools), presenting realistic retrieval and parameter schema matching challenges.
- **Perturbation Variants:** To rigorously evaluate agent robustness, the benchmark incorporates four difficulty variants: canonical tasks (127 scenarios), distraction tool injection (158 scenarios), scrambled parameter descriptors (142 scenarios), and insufficient information queries (82 scenarios) where agents must safely refuse or ask clarifying questions rather than invoking tools blindly.

Algorithm 1 DynSTEER Execution-Time Stage Evaluation and Control

Require: Public task view V_{public} , candidate count C , stop thresholds Θ

Ensure: Final evaluation report $\mathcal{R}$, termination status

```
1:  $\{G_1, \dots, G_C\} \leftarrow \text{SynthesizeCandidates}(V_{\text{public}}, C)$ 
2:  $G \leftarrow \text{ValidateAndAggregate}(\{G_1, \dots, G_C\})$  {Milestone DAG  $G = (M, E, \mathcal{K}, \mathcal{M}_{\text{fatal}})$ }
3:  $M_{\text{settled}} \leftarrow \emptyset$ ,  $t \leftarrow 0$ ,  $\mathcal{R} \leftarrow \emptyset$ ,  $\mathbf{w}_1 \leftarrow \text{InitWeights}()$ ,  $\text{Level}^{(1)} \leftarrow J_{\text{cheap}}$ 
4: for each event  $e_k$  in agent execution rollout do
5:   if  $e_k \in \mathcal{M}_{\text{fatal}}$  then
6:     return  $\text{Halt}(\mathcal{R}, \text{reason} = \text{"Fatal Minefield"}, k)$ 
7:   end if
8:   if  $e_k$  is Closed Agent Action  $a_t$  then
9:      $t \leftarrow t + 1$ 
10:     $\mathcal{F}_t \leftarrow \{m \in M \setminus M_{\text{settled}} \mid \text{Pred}(m) \subseteq M_{\text{settled}}\}$ 
11:     $m^* \leftarrow \text{MatchMilestone}(\mathcal{F}_t, a_t, s_t)$ 
12:    if  $m^* \neq \text{None}$  then
13:       $M_{\text{settled}} \leftarrow M_{\text{settled}} \cup \{m^*\}$ 
14:       $\mathcal{W} \leftarrow [t_{\text{idom}}(m^*), t]$  {Dominator-anchored evidence window}
15:       $S_t, \mathbf{s}_t, \mathbf{u}_t, \text{diag} \leftarrow \text{EvaluateStage}(\mathcal{W}, m^*, \mathbf{w}_t, \text{Level}^{(t)})$ 
16:       $\mathcal{R} \leftarrow \mathcal{R} \cup \{(m^*, S_t, \text{diag})\}$ 
17:       $\mathbf{w}_{t+1}, \text{Level}^{(t+1)} \leftarrow \text{UpdateEvaluationPolicy}(\mathbf{w}_t, \mathbf{s}_t, \mathbf{u}_t, S_t, \Theta)$  {Dynamic strategy adaptation}
18:    end if
19:    if  $\text{CheckEarlyStopCriteria}(\mathcal{R}, \mathcal{F}_t, S_t, t, \Theta)$  then
20:      return  $\text{Halt}(\mathcal{R}, \text{reason} = \text{"Early Termination"}, t)$ 
21:    end if
22:  end if
23: end for
24: return  $\text{FinalizeReport}(\mathcal{R}, M_{\text{settled}})$ 
```

- **Safety Minefields:** The dataset contains 704 explicit minefield invariants across the 509 scenarios, encompassing irreversible database deletions, privacy leaks, and invalid transaction commitments.

B ALGORITHMIC FORMULATIONS AND PROOF DETAILS

B.1 DYNSTEER EXECUTION-TIME STAGE EVALUATION AND CONTROL ALGORITHM

Algorithm 1 formalizes the complete execution-time stage evaluation, dominator evidence window extraction, uncertainty-adaptive judge routing, and early termination protocol, matching our implementation in `dynsteer/evaluate/evaluator.py` and `policy.py`.

B.2 BLUEPRINT CONSENSUS AGGREGATION DETAILS

The blueprint aggregation pipeline synthesizes multiple candidate graphs $\{G_1, \dots, G_C\}$ into a canonical Milestone DAG $G = (M, E, \mathcal{K}, \mathcal{M}_{\text{fatal}})$.

Let each candidate graph be represented by its set of nodes M_c , directed precedence edges E_c , and tool argument bindings $\mathcal{K}_c$. We define node semantic equivalence $\sim$ using canonical tool names and parameter role bindings:

$$m_i \sim m_j \iff \text{ToolName}(m_i) = \text{ToolName}(m_j) \wedge \text{Role}(m_i) = \text{Role}(m_j) \quad (11)$$

For each equivalence class $[m]$, its consensus support is:

$$\text{Support}([m]) = \sum_{c=1}^{C_{\text{valid}}} \mathbb{I}(\exists m' \in M_c \text{ s.t. } m' \in [m]) \quad (12)$$

Milestones with $\text{Support}([m]) \geq \lceil (C_{\text{valid}} + 1)/2 \rceil$ are admitted into the canonical milestone set M . Precedence edges are aggregated analogously, and transitive reduction $\text{TR}(E)$ is computed using

Tarjan’s linear-time topological ordering algorithm (Tarjan, 1972) to eliminate redundant shortcut edges.

B.3 IMMEDIATE DOMINATOR STAGE ANCHORING PROOF

Let $G = (M, E)$ be the DAG with augmented entry node s_0 . A node d dominates node n ($d \text{ dom } n$) if every directed path from s_0 to n contains d . The **immediate dominator** $\text{idom}(n)$ is the unique node that strictly dominates n without dominating any other strict dominator of n . We compute the dominator tree using the Lengauer-Tarjan algorithm (Lengauer & Tarjan, 1979) in $\mathcal{O}(|E|\alpha(|E|, |M|))$ time.

When milestone m^* is settled at step t , its stage evidence window $\mathcal{W}(m^*) = [t_{\text{idom}(m^*)}, t]$ encompasses precisely the intermediate steps executed between its immediate dominator and m^* . Because m^* could not have been reached without traversing $\text{idom}(m^*)$, all causal dependencies are preserved while concurrent, independent branches are cleanly partitioned.

C QUALITATIVE CASE STUDIES AND ERROR ANALYSIS

C.1 CASE 1: EARLY TERMINATION OF INFINITE SEARCH DEADLOCK

Scenario ID:

`search_message_with_recency_oldest_multiple_user_turn_alt_3_distraction_tools`

Agent Model: DeepSeek-V4-Pro under ReAct prompting.

Execution Trace Summary:

- **Step 1:** The user requests the oldest message from a contact. The agent correctly calls `get_current_timestamp()`, settling milestone M_1 with $S_1 = 1.0$.
- **Step 2:** The agent invokes `search_messages(creation_timestamp_lowerbound=1785839960)`. The API returns an empty list because the bound is in the future.
- **Step 3–18:** The agent enters an unrecoverable parameter loop, repeatedly querying `search_messages` with slight arbitrary perturbations of the timestamp parameter (e.g., subtracting 100, adding 50), continuously receiving empty results.
- **Native Default Evaluation:** The agent was allowed to run until the hard environment timeout of 100 turns, consuming 184.9 seconds of wall-clock time and over 32,000 tokens, only to fail at the end with a score of 0.0.
- **DynSTEER Online Behavior:** DynSTEER’s frontier monitor tracked that milestone M_2 (`retrieve_target_message`) remained unsettled across the ready frontier $\mathcal{F}_t$. At Step 18, representing 18.0% of trajectory progress, DynSTEER triggered a **Frontier Deadlock Stop**, terminating execution immediately. This saved **82.0% of wasteful execution turns** and 184.9 seconds of pipeline runtime.

C.2 CASE 2: FIRST-ERROR LOCALIZATION VS. FLUKE SUCCESS

Scenario ID:

`add_reminder_content_and_date_and_time_10_distraction_tools`

Agent Model: DeepSeek-V4-Flash.

Execution Trace Summary:

- The agent was tasked with setting a reminder with a specific date and time.
- In intermediate steps, the agent called an invalid distraction API `set_calendar_alarm` with malformed argument types, which failed.
- The agent then stumbled upon `add_reminder` after multiple trial-and-error retries.
- **Native Default Evaluation:** Scored the final state as 1.0000 for full resolution, completely ignoring the 4 erroneous intermediate API calls and high token overhead.

- **DynSTEER Stage Review:** Detected that the stage leading to milestone M_1 incurred severe tool compliance penalties and inefficiency deductions, yielding $S_1 = 0.7230$. DynSTEER correctly preserved the distinction between clean one-shot execution and inefficient trial-and-error flukes.

D PROMPT TEMPLATES FOR MILESTONE GENERATION AND STAGE JUDGES

This section documents the primary prompt templates used across DynSTEER’s pipeline, formatted in structured boxed environments.

Prompt Template 1: Milestone Graph Candidate Generation Prompt

System Prompt:

You generate candidate milestone goal graphs, not step-by-step trajectories. A node belongs in one candidate graph only when, under that candidate’s complete and feasible interpretation, every valid completion must reach that operation or goal. One candidate is a hypothesis about necessity; necessity across candidates is decided later by code aggregation.

User Prompt Input:

This is candidate batch `{batch_index}`. Focus code: `{focus_code}`.

Batch focus: `{focus_instruction}`

Core Requirements:

- Return standalone candidate graphs for the current task based strictly on the public task view `{task.view}`.
- Each graph must define its own milestones, dependency edges, dynamic bindings, and fatal minefields.
- Seek genuine diversity: explore distinct valid tools, direct public state lookups, or alternative precedence orders.
- Output must strictly follow the JSON schema specifying nodes, edges, dependencies, and minefield operations.

Prompt Template 2: Standard Judge Unified Multi-Field Prompt

System Role:

You are a strict task-trajectory evaluator. Evaluate only the supplied task, stage interval, constraint checks, and trajectory steps. Do not invent external facts.

Context Inputs:

- `stage_goal`: Success condition for the current dominator-bounded stage interval.
- `rubrics`: Target dimensions to score simultaneously (e.g., correctness, efficiency, safety, tool compliance).
- `steps`: Observed actions and tool returns strictly within the stage evidence window $\mathcal{W}(m^*)$.

Evaluation Instructions:

- Determine whether this stage satisfies `stage_goal` within the supplied interval.
- Score each target dimension from 0.0 to 1.0 based on explicit behavioral evidence.
- Provide concise diagnostic justifications citing specific step indices and tool call names.
- Output format: Strict JSON mapping dimension names to score, confidence, and diagnostic rationale.

Prompt Template 3: Expensive Judge Specialist Prompt (Tool Quality Dimension)

Specialist Objective:

You are the specialist LLM-Judge for the `tool_quality` dimension. Score only tool-use quality.

Detailed Forensic Focus:

- Whether the agent selected the optimal tools for the active stage goal.
- Whether tool arguments are complete, accurate, and type-conforming without hallucinations.
- Whether return values, exceptions, and empty results were correctly parsed and handled rather than ignored.
- Penalize redundant retry loops, irrelevant exploratory calls, and unhandled tool exceptions.
- Output: Calibrated continuous score in $[0, 1]$, confidence score, and forensic audit trace.